

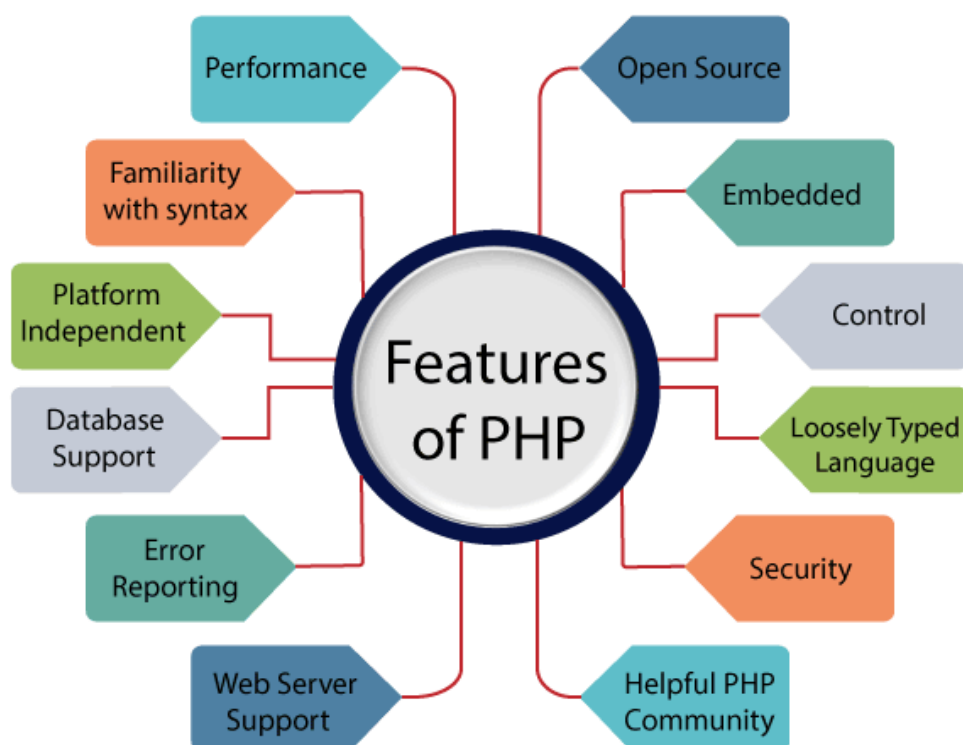
TECHNOLOGY LEARNT

PHP

PHP (Hypertext Preprocessor) is a widely-used server-side scripting language designed for web development. It is known for its ease of integration with HTML and its ability to handle dynamic content.

Key features and aspects of PHP:

- **Server-Side Scripting:** Executes code on the server to generate dynamic web content.
- **Database Integration:** Easily connects to databases like MySQL and NoSQL databases such as DynamoDB.
- **Embedded in HTML:** Allows embedding PHP code within HTML files.
- **Cross-Platform:** Runs on various operating systems, including Windows, Linux, and macOS.
- **Rich Ecosystem:** Supports numerous frameworks and libraries for enhanced functionality.



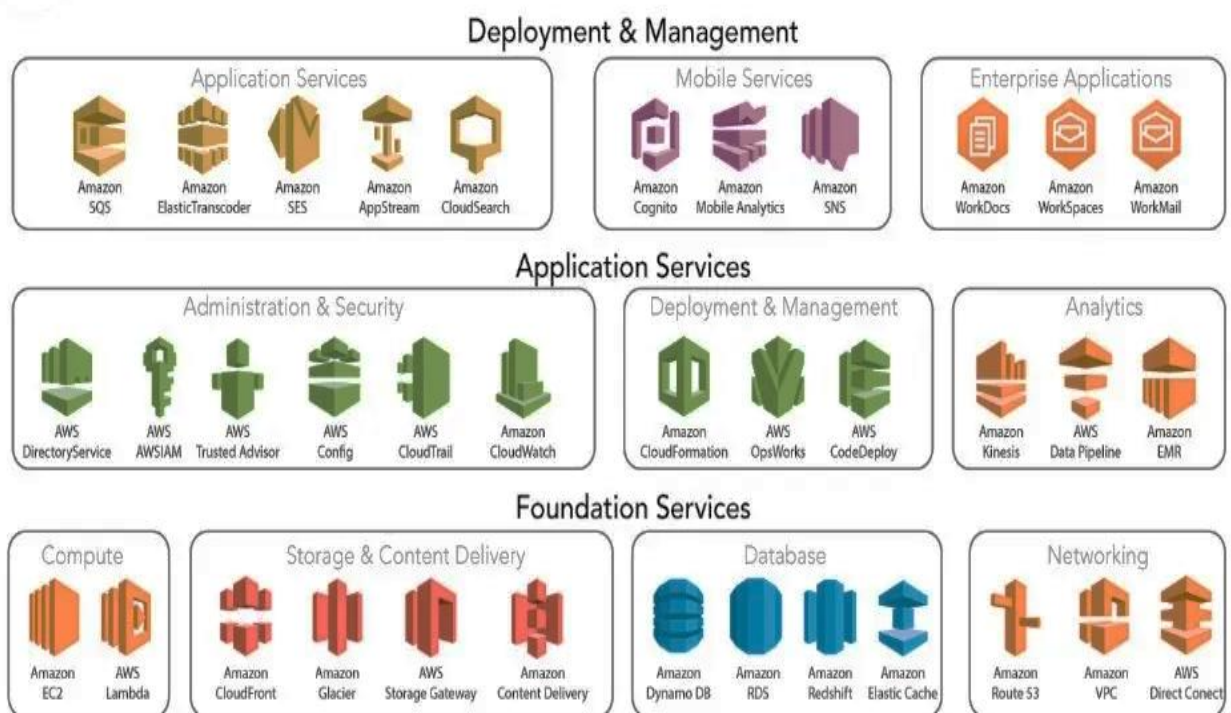
AMAZON WEB SERVICES (AWS)

AWS is a comprehensive cloud computing platform offered by Amazon, providing a vast array of cloud-based services for computing, storage, and data management. It is widely recognized for its scalability, flexibility, and robust set of features.

Key characteristics and features of AWS:

- **Scalability and Flexibility:** AWS provides scalable cloud computing resources that can be adjusted based on demand, allowing applications to handle varying workloads efficiently. Services like AWS EC2 (Elastic Compute Cloud) enable users to scale their computing capacity seamlessly.
- **Global Infrastructure:** AWS operates a vast network of data centers worldwide, ensuring high availability and redundancy. This global infrastructure supports low-latency access and provides disaster recovery solutions.
- **Security and Compliance:** AWS incorporates robust security features, including data encryption, identity and access management (IAM), and compliance with various industry standards and regulations.
- **Pay-as-You-Go Pricing:** AWS uses a flexible pricing model where users only pay for the resources they use, without upfront costs. This pay-as-you-go approach allows for cost-effective management of cloud resources.
- **High Availability and Disaster Recovery:** AWS provides tools and services to ensure that applications are resilient and can recover from failures. Services like AWS Elastic Load Balancing and AWS Route 53 help maintain high availability and ensure reliable application performance.
- **Integrated Ecosystem:** AWS includes a broad range of integrated tools and services, such as AWS CloudFormation for infrastructure management, AWS Lambda for serverless computing, and AWS CloudWatch for monitoring and logging. This integrated ecosystem simplifies the development and management of cloud-based applications.

- **Advanced Analytics and AI Services:** AWS offers powerful analytics and machine learning services, including Amazon SageMaker for building and deploying machine learning models, and Amazon Redshift for data warehousing. These services enable advanced data analysis and the creation of intelligent applications.
- **Developer and DevOps Tools:** AWS provides a comprehensive suite of tools for development and operations, including AWS CodeBuild, AWS CodeDeploy, and AWS CodePipeline for continuous integration and delivery (CI/CD). These tools streamline the development process, automate deployment, and enhance collaboration between development and operations teams.



AWS SDK FOR PHP

The AWS SDK for PHP is a powerful library that facilitates the interaction between PHP applications and AWS services, streamlining the integration and management of cloud-based resources.

Key features and aspects of AWS SDK:

- **API Integration:** Provides comprehensive APIs for accessing various AWS services, including DynamoDB, S3, and EC2. This includes methods for CRUD operations, service management, and resource configuration.
- **Ease of Use:** Abstracts the complexity of AWS service APIs, enabling developers to interact with AWS services using straightforward PHP methods. This reduces the need for low-level API management and simplifies code.
- **Configuration Management:** Supports detailed configuration through AWS credentials, region settings, and service-specific parameters. Developers can manage AWS access keys, secret keys, and region settings either programmatically or through configuration files.
- **Documentation and Examples:** Offers extensive documentation, including tutorials, API references, and example code snippets, to assist developers in implementing AWS services. This helps in understanding best practices and optimizing usage.
- **Error Handling and Retries:** Incorporates built-in error handling and automatic retry mechanisms to manage and recover from API call failures, network issues, and service interruptions.
- **Asynchronous Requests:** Supports asynchronous operations, allowing PHP applications to handle long-running processes and improve performance by not blocking the execution flow.
- **Version Management:** Regularly updated with new features, enhancements, and security improvements. Ensures compatibility with the latest AWS services and PHP versions.
- **Security and Compliance:** Includes built-in support for AWS security features, such as encryption, IAM roles, and VPC configurations, ensuring secure interactions with AWS resources.

AWS DYNAMODB

AWS DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability. It is used for managing and storing data in a flexible schema.

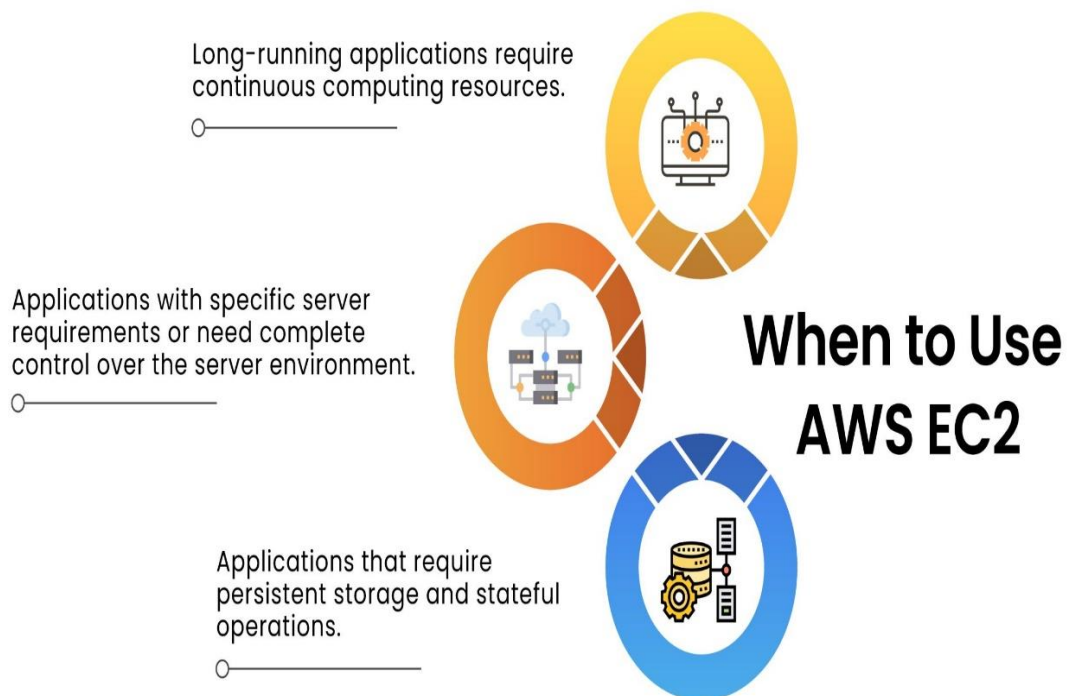
Key features and aspects of AWS DynamoDB:

- **Fully Managed:** Handles all aspects of database management, including hardware provisioning, patching, backups, and scaling. Reduces operational overhead and allows developers to focus on application development.
- **Scalability:** Automatically adjusts to handle varying data loads and traffic patterns. Provides both on-demand and provisioned capacity modes, allowing users to scale resources based on application needs.
- **Low Latency:** Delivers single-digit millisecond response times for read and write operations, making it suitable for high-performance applications such as gaming, IoT, and real-time analytics.
- **Flexible Data Model:** Supports both key-value and document data models. Enables the storage of diverse data types, including JSON documents, and provides flexibility in schema design.
- **Global Tables:** Facilitates multi-region replication, ensuring data availability and durability across different geographical locations. Supports global applications with low-latency access.
- **Provisioned and On-Demand Capacity:** Offers provisioned capacity for predictable workloads and on-demand capacity for variable workloads, allowing users to optimize cost and performance.
- **Integrated Security:** Provides robust security features, including encryption at rest, encryption in transit, IAM policies for fine-grained access control, and VPC endpoints for secure connections.
- **Query and Indexing:** Enables advanced querying capabilities with primary keys, secondary indexes (both global and local), and supports efficient data retrieval and filtering.

- **Streams:** Offers DynamoDB Streams to capture changes to items in a table and respond to these changes in real-time. Useful for event-driven architectures and integration with other services.

AWS EC2

AWS EC2 (Elastic Compute Cloud) is a scalable virtual server service that provides flexible compute capacity in the cloud, enabling users to run applications and manage workloads with high availability and performance.



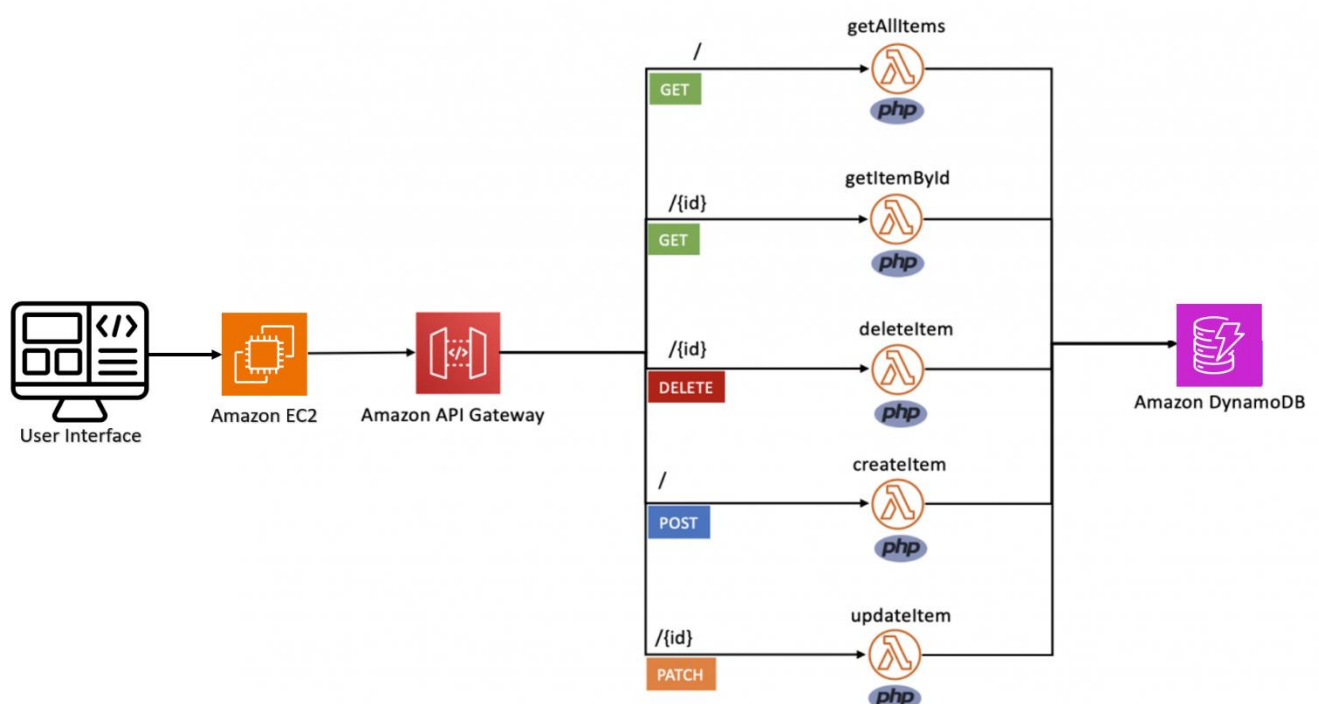
Key features and aspects of AWS EC2:

- **Scalability:** Allows dynamic scaling of compute resources based on demand. Provides auto-scaling capabilities to adjust the number of instances based on traffic and application needs.
- **Instance Types:** Offers a variety of instance types optimized for different use cases, including compute-optimized, memory-optimized, storage-optimized, and GPU instances for specific workloads.

- **Elastic Load Balancing:** Distributes incoming application traffic across multiple EC2 instances to improve availability and fault tolerance. Supports both Application Load Balancer (ALB) and Network Load Balancer (NLB) for different use cases.
- **Integration with AWS Services:** Seamlessly integrates with other AWS services such as RDS for managed databases, S3 for object storage, and CloudWatch for monitoring and logging.
- **Storage Options:** Provides options for persistent storage using Elastic Block Store (EBS) and temporary storage with instance store volumes. EBS supports various volume types, including SSD and HDD.

INTEGRATION OF PHP WITH AWS DYNAMODB

In this project, the most critical integration is between PHP and AWS DynamoDB. PHP, serving as the server-side scripting language, utilizes the AWS SDK for PHP to interact with DynamoDB. This integration allows PHP to perform key database operations such as adding, updating, and deleting expense records. The AWS SDK for PHP provides a set of API methods that facilitate seamless communication between PHP scripts and DynamoDB.



HTML

HTML (HyperText Markup Language) is the standard markup language used to create and structure web pages. It defines the elements and layout of a web page.

Key features and aspects of HTML:

- **Markup Language:** Uses a hierarchical system of tags to structure content on web pages. Tags such as `<html>`, `<head>`, `<body>`, and `<footer>` define the structure and layout of web content.
- **Elements and Attributes:** Provides a wide range of elements, including block-level elements (`<div>`, `<p>`, `<header>`) and inline elements (`<span>`, `<a>`, `<strong>`). Attributes such as `id`, `class`, and `href` customize the behavior and appearance of elements.
- **Semantic HTML:** Utilizes semantic tags like `<article>`, `<section>`, and `<aside>` to define the meaning and role of content, improving accessibility and search engine optimization (SEO).
- **Forms and Input Elements:** Supports the creation of interactive forms using elements like `<form>`, `<input>`, `<select>`, and `<button>`. Includes form attributes for validation and data submission.
- **Linking and Embedding:** Allows the inclusion of external resources and media on web pages using `<link>`, `<script>`, and `<embed>`. Facilitates linking to stylesheets, scripts, and multimedia content.
- **Accessibility Features:** Includes features to enhance accessibility, such as `alt` attributes for images, `aria` roles for screen readers, and semantic HTML to provide meaningful context.
- **Global Attributes:** Supports global attributes like `data-*`, `title`, and `lang` that can be applied to any HTML element to provide additional information and improve usability.

CSS

CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation and layout of HTML elements. It controls the appearance of web pages.

Key features and aspects of CSS :

- **Styling:** Defines the visual style of HTML elements, including properties for colors, fonts, text alignment, spacing, and layout. CSS rules apply styles based on selectors and declarations.
- **Selectors and Specificity:** Uses a variety of selectors to target HTML elements, including class selectors (.classname), ID selectors (#idname), and attribute selectors ([type="text"]). Specificity determines which styles are applied when multiple rules match an element.
- **Responsive Design:** Implements media queries to create responsive and adaptive designs. Adjusts the layout and styling based on screen size, device orientation, and other factors to ensure a consistent user experience across devices.
- **Flexbox and Grid Layouts:** Provides advanced layout techniques such as Flexbox and CSS Grid for creating complex and flexible layouts. Flexbox manages layout along a single axis, while Grid offers a two-dimensional layout system.
- **Animation and Transitions:** Supports animations and transitions to enhance user interactions and visual effects. Uses properties like transition, animation, and keyframes to create dynamic and engaging user experiences.
- **Preprocessing Support:** Integrates with CSS preprocessors like SASS and LESS, which offer features such as variables, mixins, nesting, and functions to streamline and extend CSS.

PROJECT WORK

EXPENSE TRACKER USING AWS EC2 AND DYNAMODB

The main aim of designing and developing this Expense Tracker application is to provide users with a secure, efficient, and user-friendly platform to manage their personal expenses. This project leverages PHP, AWS SDK for PHP, and AWS DynamoDB to create a web application where users can add, update, view, and delete expenses. The application ensures data integrity and security, offering a seamless user experience.

MAIN PURPOSE

The traditional method of maintaining expense records involved manually entering and recording transactions, which could be tedious and error-prone. With this Expense Tracker application, users can easily manage their expenses online, providing a more efficient and accurate way to track their spending. This project offers a practical understanding of integrating cloud services with web applications, enhancing scalability and performance.

MAIN GOAL

- **Efficiency:** Develop a software program that simplifies expense management, ensuring all transactions are recorded accurately and efficiently.
- **Customer Satisfaction:** Provide users with a comfortable and risk-free experience, protecting their privacy and data.
- **Time-Saving:** Enable users to manage their expenses without the need for manual record-keeping.
- **Data Protection:** Ensure users' data, including their expenses and personal information, is secure.

MODULES

1. Expense Management Module

This module is designed for users to manage their personal expenses. It includes functionalities for adding, updating, viewing, and deleting expenses.

- **Add Expense:**

Users can add new expenses by entering details such as description, amount, category, and date. The application uses AWS DynamoDB to store expense data securely.

- **View Expenses:**

Users can view a list of all their expenses, with details such as description, amount, category, and date. The total amount of expenses is calculated and displayed.

- **Update Expense:**

Users can update existing expenses by modifying details such as description, amount, category, and date.

- **Delete Expense:**

Users can delete existing expenses from the record.

2. Expense Form Interface Module

This module is responsible for capturing expense details from the user through a structured form.

- **Form Design:**

Includes an HTML form designed to capture detailed expense information from the user.

- **Input Fields:**

Provides input fields for ID, description, amount, category, and date, ensuring comprehensive data collection.

- **Action Buttons:**

Incorporates buttons for adding, updating, or deleting expenses, facilitating efficient expense management.

3. Visual Styling and Responsiveness Module

This module ensures the application interface is aesthetically pleasing and user-friendly.

- **CSS Styling:**

Utilizes CSS to style the form and expense list, enhancing the visual appeal and usability of the application.

- **Responsive Design:**

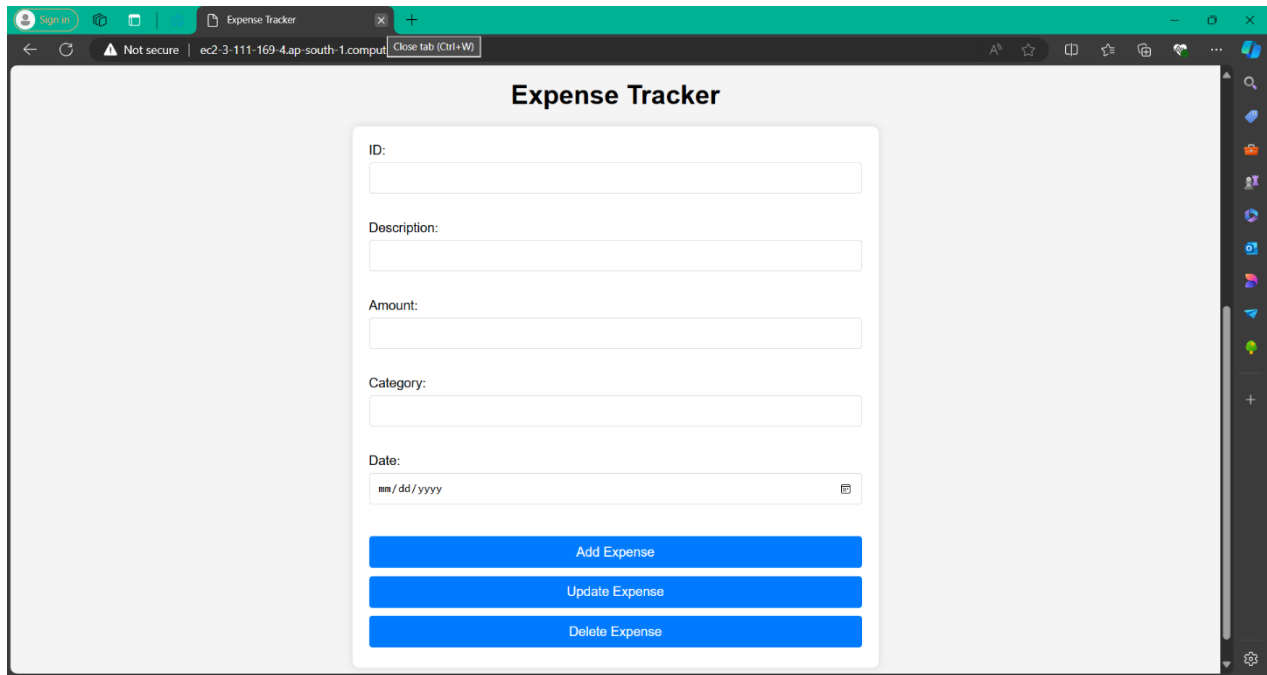
Ensures the application is accessible and functional across various devices, offering a seamless user experience.

- **UI Elements:**

Includes styles for alert messages, form layout, buttons, and expense list items, contributing to a cohesive and intuitive interface.

In conclusion, a well-designed Expense Tracker application with robust Expense Management, Expense Form Interface, and Visual Styling and Responsiveness Modules is essential for modern personal finance management. These modules enable seamless addition, update, and deletion of expense records, improving operational efficiency and data integrity.

OUTPUT



Expense Tracker

ID:

Description:

Amount:

Category:

Date:

mm/dd/yyyy

Add Expense

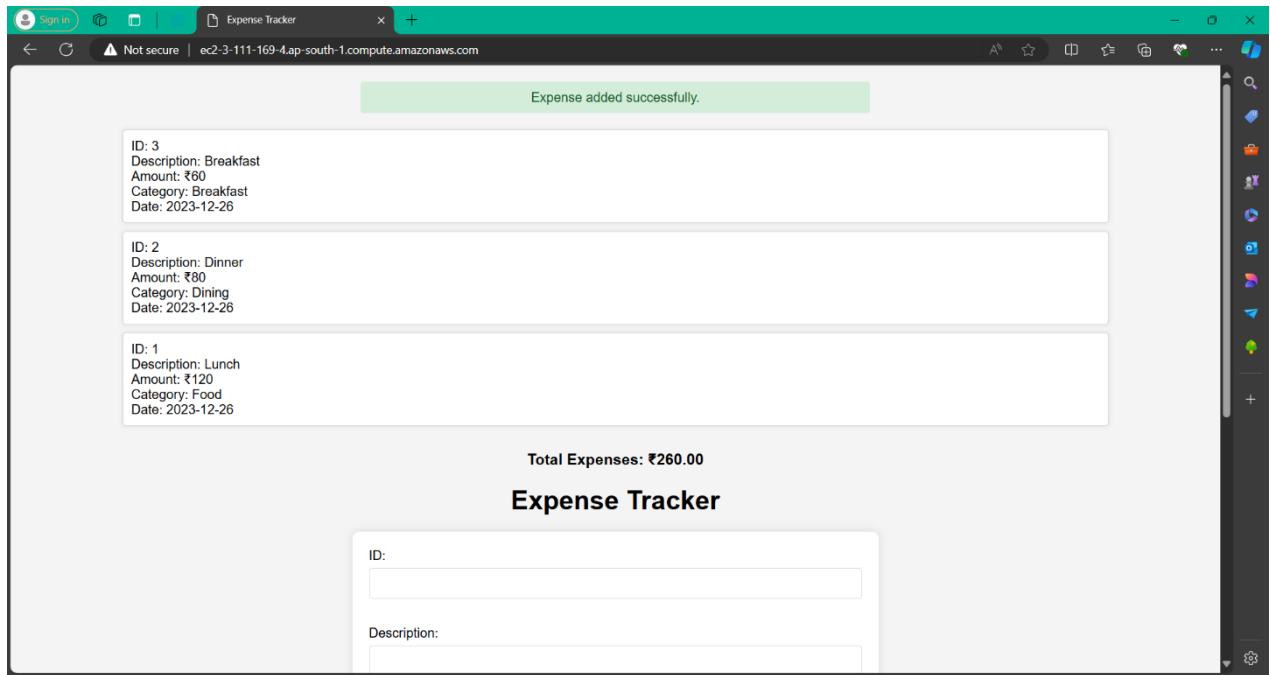
Update Expense

Delete Expense

(a) Web Application Interface

This image focuses on the expense entry form part of the application. It shows:

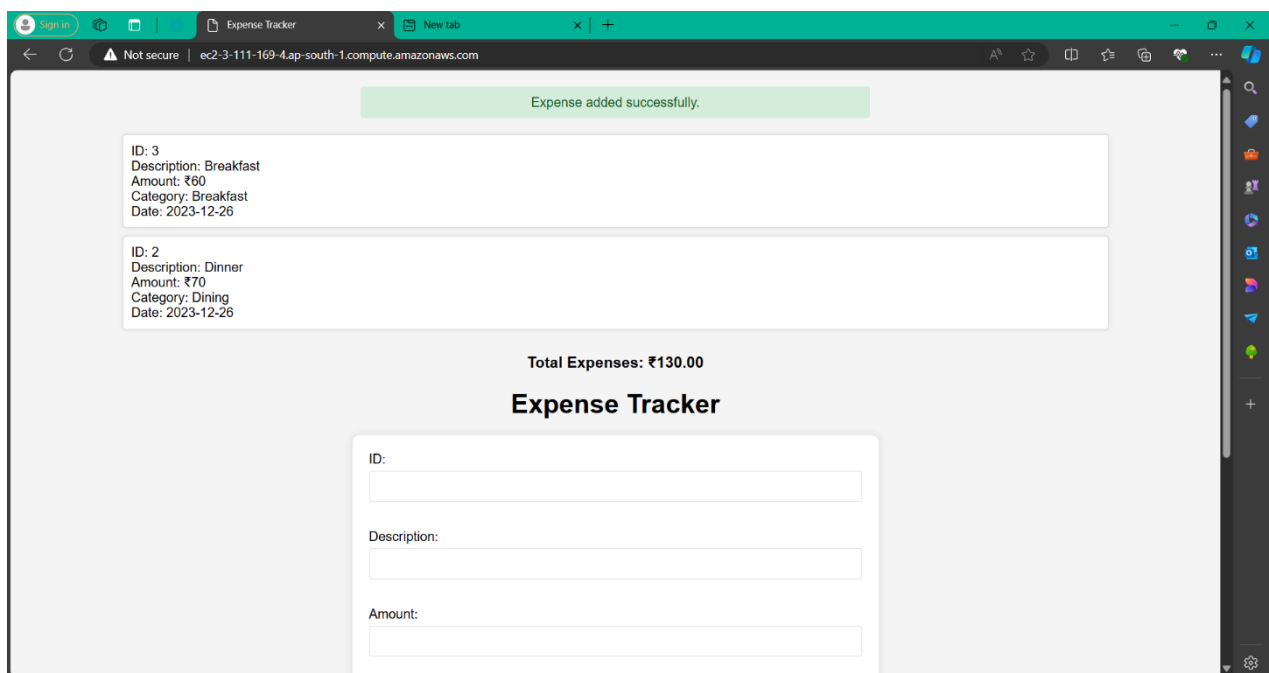
- The "Expense Tracker" title at the top.
- The expense entry form with fields for:
 - ID
 - Description
 - Amount in Indian Rupees (₹)
 - Category
 - Date (with a date picker interface)
- Three action buttons: "Add Expense", "Update Expense", and "Delete Expense".



(b) Adding expenses in the application

This image shows the application after expenses have been added. It displays:

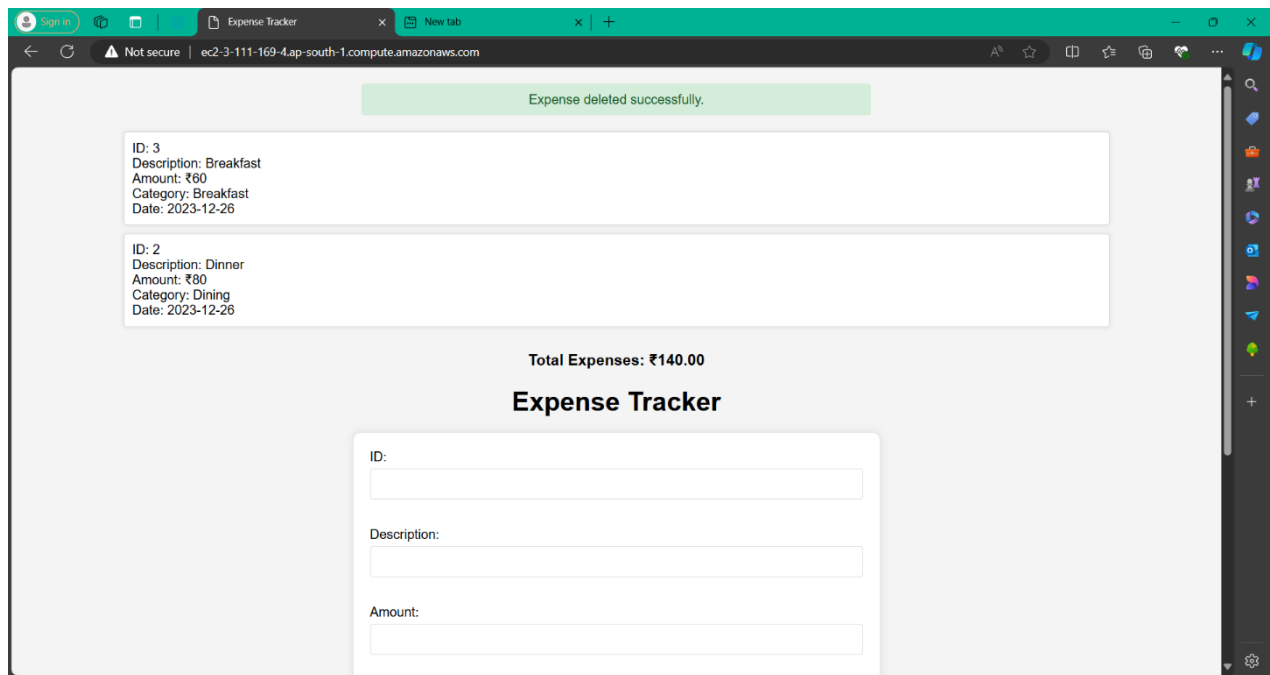
- A green success message: "Expense added successfully."
- A total of three expenses has been added that can be seen in the application.
- An updated total expenses amount of ₹260.00 is displayed.



(c) Deleting an expense from the record

This image shows the application after an expense has been deleted. It displays:

- A green success message: "Expense deleted successfully."
- Two remaining expense entries are present.
- An updated total expenses amount of ₹130.00 is displayed.



(d) Updating an expense from the record

This image shows the application after an expense has been updated. It displays:

- A green success message: "Expense added successfully."
- An expense already present in the record has been updated.
- An updated total expenses amount of ₹140.00 is displayed.

CODING

index.php

```
<?php
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
```

```
error_reporting(E_ALL);
```

```
require 'vendor/autoload.php';
```

```
use Aws\DynamoDb\DynamoDbClient;
```

```
use Aws\Exception\AwsException;
```

```
// Initialize DynamoDB Client
```

```
$client = new DynamoDbClient([
```

```
    'region' => 'ap-south-1',
```

```
    'version' => 'latest',
```

```
]);
```

```
$tableName = 'tasks';
```

```
// Function to add an expense
```

```
function addExpense($client, $tableName, $id, $description, $amount, $category,  
$date) {
```

```
    try {
```

```
        $result = $client->putItem([
```

```
            'TableName' => $tableName,
```

```
            'Item' => [
```

```
                'id' => ['N' => (string)$id],
```

```
                'description' => ['S' => $description],
```

```
                'amount' => ['N' => (string)$amount],
```

```
                'category' => ['S' => $category],
```

```
                'date' => ['S' => $date],
```

```
            ],
```

```
        ];
```



```

        echo "<p class='alert success'>Expense added successfully.</p>";
    } catch (AwsException $e) {
        echo "<p class='alert error'>Unable to add expense. Error: " . $e-
>getMessage() . "</p>";
    }
}

```

// Function to view expenses

```

function viewExpenses($client, $tableName) {
    try {
        $result = $client->scan([
            'TableName' => $tableName,
        ]);
        echo "<div class='expense-list'>";
        $total = 0;
        foreach ($result['Items'] as $item) {
            $id = $item['id']['N'];
            $description = $item['description']['S'];
            $amount = $item['amount']['N'];
            $category = $item['category']['S'];
            $date = $item['date']['S'];
            $total += (float)$amount;
            echo "<div class='expense-item'>";
            echo "<div>ID: $id</div>";
            echo "<div>Description: $description</div>";
            echo "<div>Amount: ₹$amount</div>";
            echo "<div>Category: $category</div>";
            echo "<div>Date: $date</div>";
            echo "</div>";
        }
    } catch (Exception $e) {
        echo "<p>Error: " . $e->getMessage() . "</p>";
    }
}

```

```

    }
    echo "</div>";
    echo "<h3>Total Expenses: ₹" . number_format($total, 2) . "</h3>";
} catch (AwsException $e) {
    echo "<p class='alert error'>Unable to fetch expenses. Error: " . $e-
>getMessage() . "</p>";
}
}

// Function to update an expense
function updateExpense($client, $tableName, $id, $description, $amount,
$category, $date) {
    try {
        $result = $client->updateItem([
            'TableName' => $tableName,
            'Key' => [
                'id' => ['N' => (string)$id],
            ],
            'UpdateExpression' => 'SET description = :desc, amount = :amt, category =
:cat, date = :dt',
            'ExpressionAttributeValues' => [
                ':desc' => ['S' => $description],
                ':amt' => ['N' => (string)$amount],
                ':cat' => ['S' => $category],
                ':dt' => ['S' => $date],
            ],
        ]);
        echo "<p class='alert success'>Expense updated successfully.</p>";
    } catch (AwsException $e) {

```

```

        echo "<p class='alert error'>Unable to update expense. Error: " . $e-
>getMessage() . "</p>";
    }
}

// Function to delete an expense
function deleteExpense($client, $tableName, $id) {
    try {
        $result = $client->deleteItem([
            'TableName' => $tableName,
            'Key' => [
                'id' => ['N' => (string)$id],
            ],
        ]);
        echo "<p class='alert success'>Expense deleted successfully.</p>";
    } catch (AwsException $e) {
        echo "<p class='alert error'>Unable to delete expense. Error: " . $e-
>getMessage() . "</p>";
    }
}

// Handle form submissions
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (isset($_POST['add'])) {
        addExpense($client, $tableName, $_POST['id'], $_POST['description'],
$_POST['amount'], $_POST['category'], $_POST['date']);
    } elseif (isset($_POST['update'])) {
        updateExpense($client, $tableName, $_POST['id'], $_POST['description'],
$_POST['amount'], $_POST['category'], $_POST['date']);
    }
}

```

```

    } elseif (isset($_POST['delete'])) {
        deleteExpense($client, $tableName, $_POST['id']);
    }
}
viewExpenses($client, $tableName);
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Expense Tracker</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>Expense Tracker</h1>

    <form method="POST" class="expense-form">
        <label for="id">ID:</label>
        <input type="number" name="id" id="id" required><br>
        <label for="description">Description:</label>
        <input type="text" name="description" id="description" required><br>
        <label for="amount">Amount:</label>
        <input type="number" name="amount" id="amount" step="0.01"
required><br>
        <label for="category">Category:</label>
        <input type="text" name="category" id="category" required><br>
        <label for="date">Date:</label>
        <input type="date" name="date" id="date" required><br>

```

```
<button type="submit" name="add">Add Expense</button>
<button type="submit" name="update">Update Expense</button>
<button type="submit" name="delete">Delete Expense</button>
</form>

<div class="expense-list">
  <!-- Expenses will be displayed here -->
</div>
</body>
</html>
```

styles.css

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
  padding: 0;
  background-color: #f4f4f4;
}
```

```
h1 {
  text-align: center;
  margin-bottom: 20px;
}
```

```
.expense-form {
  display: flex;
  flex-direction: column;
  max-width: 600px;
```

```
margin: 0 auto;
padding: 20px;
background-color: #fff;
border-radius: 8px;
box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
```

```
.expense-form label {
  margin-bottom: 5px;
}
```

```
.expense-form input {
  margin-bottom: 15px;
  padding: 10px;
  width: 100%;
  border: 1px solid #ddd;
  border-radius: 4px;
  box-sizing: border-box;
}
```

```
.expense-form button {
  margin: 5px 0;
  padding: 10px 20px;
  border: none;
  color: #fff;
  background-color: #007bff;
  cursor: pointer;
  border-radius: 4px;
  font-size: 16px;
}
```

```
}
```

```
.expense-form button:hover {  
    background-color: #0056b3;  
}
```

```
.expense-list {  
    max-width: 600px;  
    margin: 20px auto;  
}
```

```
.expense-item {  
    border: 1px solid #ddd;  
    padding: 10px;  
    margin-bottom: 10px;  
    border-radius: 4px;  
    background-color: #fff;  
    box-shadow: 0 0 5px rgba(0, 0, 0, 0.1);  
}
```

```
.alert {  
    padding: 10px;  
    border-radius: 4px;  
    margin-bottom: 10px;  
    max-width: 600px;  
    margin: 10px auto;  
    text-align: center;  
}
```

```
.alert.success {  
  background-color: #d4edda;  
  color: #155724;  
}
```

```
.alert.error {  
  background-color: #f8d7da;  
  color: #721c24;  
}
```